Use Eager Loading To Reduce Queries

To fix N+1 queries, use Eager Loading and eagerly load associated data. You'll need to test to make sure you get the same query results. Usually one query is more efficient even if it takes a little longer, but you'll need to test a bit to confirm.

Active Record Associations (See: *Types of Associations*[5]) like has_one, has_many, or belongs_to describe different types of relationships and generate different SQL queries to join rows of data from tables together.

Models can be configured for one of six supported types. When you see repeated queries for the same table there's a good chance all of the data can be queried for that table in one query.

Preload data using the .preload()[6] method to combine multiple queries into a single statement.

In bin/rails console, run the same code again but add .preload().

```
Vehicle.preload(:vehicle_reservations).all.each do |vehicle|
  vehicle.vehicle_reservations.size
end; nil
```

This code avoids the N+1 query pattern but it also adds one more trick. .count() was replaced with .size() which means that when all of the Vehicle Reservations were loaded at once for all of the Vehicles, the size of the loaded collection was used instead of issuing a SQL COUNT() query to count how many there were.[7]

Preloading isn't the only method of Eager Loading data. Read on to learn more.

Eager Loading With .includes()

Besides .preload(), includes() is a second technique to Eager Load data and performs an equivalent SQL query.

Try running the following query with .includes() and compare the SQL queries.

```
Vehicle.includes(:vehicle_reservations).each do |vehicle|
  vehicle.vehicle_reservations.size
end; nil
```

5. https://guides.rubyonrails.org/association_basics.html#the-types-of-associations
6. https://api.rubyonrails.org/classes/ActiveRecord/QueryMethods.html#method-i-preload
7. https://www.speedshop.co/2019/01/10/three-activerecord-mistakes.html